From Prompts to Programs

Executing Structured Natural-Language Programs on ARC-AGI Grid Puzzles

Zihuan (Ken) Jiang · Supervised by Prof. Scott Sanner

Faculty of Applied Science and Engineering · University of Toronto · 2026

What is ARC-AGI

A benchmark for human-like reasoning.

TASK FORMAT

Infer the hidden rule from **2–5 input/output examples**.
Apply the rule to a new test grid.

GRID REPRESENTATION

Small colored grids, up to **30×30 with 10 colors**.
Represent as matrices.

EVALUATION

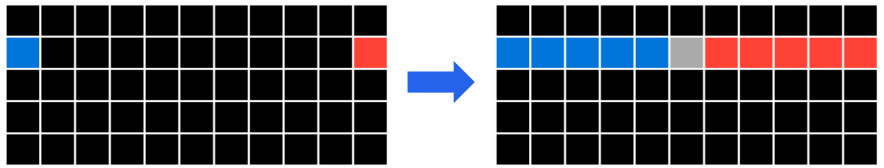
Exact match only — **every cell must be correct.**

BENCHMARK SCALE

ARC-AGI-1: 400 training / 400 evaluation / 200 private test tasks
ARC-AGI-2: 1000 training / 120 evaluation / 240 private test tasks

Task Pipeline

EXAMPLE AT A GLANCE



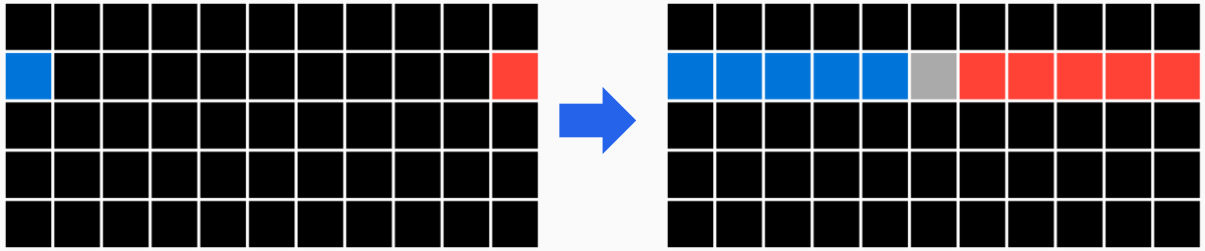
WHY IS THIS HARD?

Each task has a new rule.
Success requires abstraction, not memorization.

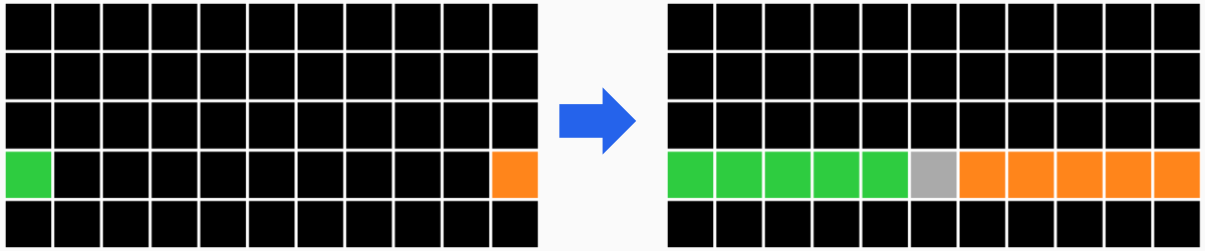
Example Task: 29c11459

How would you solve this?

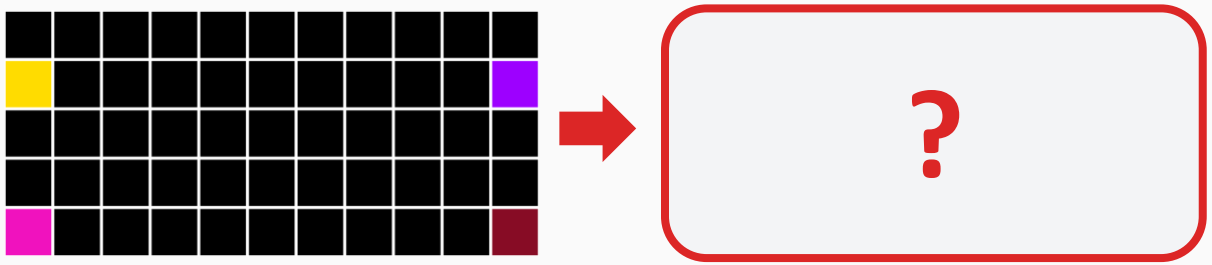
TRAIN PAIR 1



TRAIN PAIR 2



TEST (PREDICT THE OUTPUT)



INFERRED RULE

For each row,

if the row has two colored endpoints (non-black),

fill the left half with the left endpoint color,

fill the right half with the right endpoint color,

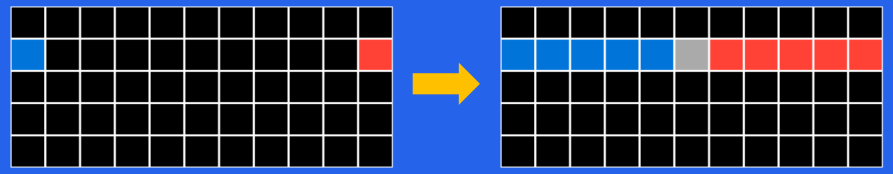
place a gray cell at the midpoint.

Rows with all-black endpoints stay unchanged.

But how should an AI system represent and execute this rule?

Approaches to Solve ARC Tasks

Three paradigms, each with fundamental trade-offs



Monolithic NL Programs

LLM writes a single paragraph of natural-language instructions.

STRENGTHS

✓ Flexible and expressive

✓ Fits LLM language priors

LIMITATIONS

✗ Cannot execute deterministically

✗ Ambiguous variable references

✗ Impossible to debug step-by-step

```
For each row, if the row has two colored
endpoints (non-black), fill the left half
with the left endpoint color, …
```

Step-by-Step NL Programs

LLM writes and executes numbered steps in natural language.

STRENGTHS

✓ More transparent reasoning

✓ Easier to inspect and debug

LIMITATIONS

✗ Implicit variable bindings

✗ More failure points per step

```
1. If a row starts with a non-black color,
note the left and right color.
2. Fill the left half with the left color.
3. Fill the right half with the right color.
4. Place gray in the middle.
```

Python Programs

LLM generates a Python function that transforms input → output.

STRENGTHS

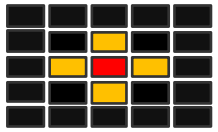
✓ Deterministic execution

✓ Each line verifiable

LIMITATIONS

✗ Lack of expressiveness

✗ Can only deal with 'rigid-grid representation'



```
def solve(grid):
  mid = len(grid[0]) // 2
  for r in range(len(grid)):
    if grid[r][0] != 0:
      for c in range(mid):
        grid[r][c] = grid[r][0]
  return grid
```

The Structure–Scale Trade-off

Why none of the existing approaches are enough

✗ MONOLITHIC PROMPTING

Flexible but unverifiable. Cannot execute deterministically. Needs very large models to faithfully execute.

✗ STEP-BY-STEP PROMPTING

More transparent, but implicit variable bindings.

✗ PYTHON PROGRAMS

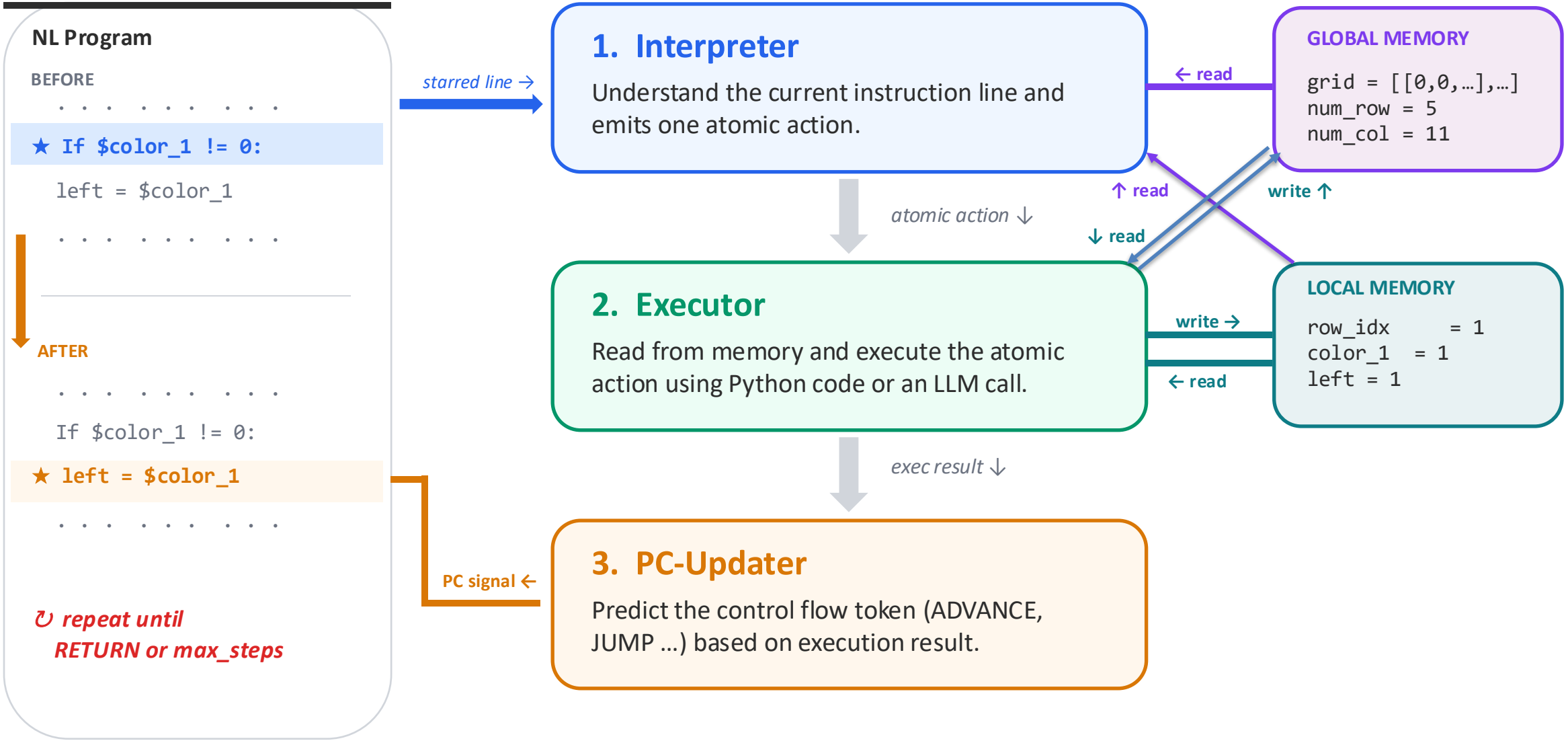
Precise and executable, but lack of expressiveness. Can only deal with rigid-grid-representation.

RESEARCH QUESTION

Can we design a structured natural-language program representation that is executable and verifiable like code, yet flexible and expressive enough to express diverse ARC transformation rules?

System Architecture: The Execution Loop

One step of the execution loop



v0 → v1: The Interpreter

From free-form steps to constrained, variable-bound actions

v0 Step-by-Step NL (baseline)

```
1. If a row starts with a non-black color, note the left and right
color.
2. Fill the left half with the left color.
3. Fill the right half with the right color.
4. Place gray in the middle.
```

✗ **"Where is "left color" stored?"** → *Variable storage*

✗ **"What is "left half"?"** → *Ambiguous reference, implicit variable*

✗ **"How is history managed?"** → *History management*

v1 Interpreter: constrained action mapping with variable memory

```
NL instruction line:
————————————————————
color_1 = <the color of
  the first cell in
  row $row>

grid[$row][$left] =
  $color_1
```

```
        Atomic action:
————————————————————
eval_query(
  dst='color_1',
  query='the color of the
    first cell in row $row')

grid_set(x='$row',
  y='$left', rhs='$color_1')
```

WHAT THE INTERPRETER SOLVES

→ **Variable memory**: variable is stored externally from LLM chat history.

→ **Explicit $-variables**: every variable reference is bound ($row, $color_1).

→ **Constrained action set**: unconstrained NL is mapped to a finite set of typed actions.

→ **JSON-style communication**: actions carry structured arguments (dst, query, expr) enabling the executor to parse and execute reliably.

CONSTRAINED ACTION SPACE

`set_local(name, val)`	Bind a local variable
`eval_expr(dst, expr)`	Evaluate arithmetic ($-aware)
`eval_condition(cond)`	Boolean with $-refs
`eval_query(dst, query)`	Spatial question → value
`grid_set(x, y, rhs)`	Write one cell
`structural / return`	Control flow / terminate

v1 outcome: Every NL instruction line is mapped to exactly one typed, $-variable-bound action — bridging natural language and deterministic execution.

v1 → v2: The Executor

Separating global memory · moving grid updates to Python

v1 Problem: LLM handles everything

- ✗ **All actions go through the LLM** — slow and error-prone.
- ✗ **Grid access errors**: LLM misreads cell values, miscounts rows/columns.
- ✗ **Grid update errors**: LLM writes wrong values to wrong positions.
- ✗ **No separation**: global memory (grid) and execution state (variables) are tangled.

v2 Solution: Hybrid Python + LLM executor

PYTHON PATH — deterministic, no LLM

Moved to Python because:

→ Variable assignment is arithmetic — no reasoning needed.

→ grid set is the CORE solving operation — must be exact.

→ Condition evaluation on $-variables is a direct expression.

```
# eval_expr:
left = $left + 1
→ local_vars['left'] = 3 + 1
= 4

# grid_set (CORE):
grid[$row][$left] = $color_1
→ grid[1][2] = 1  (exact)
```

Actions:

`set_local · eval_expr · eval_condition · grid_set · write_global · structural · return`

V2: DECOUPLED MEMORY & STATE

Global Memory

```
grid = [[0,0,…],…]
num_row = 5
num_col = 11
```

Local Memory

```
row_idx     = 1
left_color  = 1
right_color = 2
```

Why separate? Interpreter sees memory keys (not values) — cannot hallucinate grid content. Executor writes grid via Python — no LLM involved in the critical path.

LLM PATH — spatial reasoning only

eval_query: ask spatial questions
"What is the color of the first cell in row $row?"
→ LLM inspects grid → returns value

When is LLM needed?
→ Questions requiring spatial perception
→ Complex conditions the interpreter cannot reduce to arithmetic

LLM VERIFICATION LOOP

1. **Executor LLM** — receives action + grid + variable snapshot
2. **Returns JSON** — {"result": "1", "type": "color"}
3. **Verifier LLM** — checks: is this answer correct given the grid?
4. **If incorrect** — retry with feedback (up to max_retries)

v2 outcome: ~70% of actions run in Python. LLM is used only for spatial perception queries, with a verification safety net.

v2 → v3: The PC Updater

Separating control-flow reasoning from action generation

v2 Problem: Interpreter does too much

v2 LOOP (PROBLEMATIC):

Interpreter → Executor

result fed BACK to Interpreter

✗ Information overload	Interpreter receives executor result + must decide **BOTH the next control flow AND the next atomic action**.
✗ Hallucination risk	**Too much context** causes the LLM to **skip steps**, **jump to wrong lines**, or **generate actions for future lines**.
✗ Violated separation	**One module doing two jobs** (control flow + action generation) contradicts the principle of single responsibility.

v3 Solution: Dedicated PC Updater module

v3 LOOP (CLEAN SEPARATION):

Interpreter → Executor → PC Updater

generates action · *executes action* · *decides next line*

Each module does exactly one job:
Interpreter → "what action?" · Executor → "do the action" · PC Updater → "what line next?"
No module sees more information than it needs.

PC UPDATER: LLM + PYTHON HYBRID

LLM ROLE — control-flow decision only

Given the executor result and the current line context, the LLM outputs exactly ONE discrete control token from a constrained set:

ADVANCE_1	Proceed to the next line
JUMP_TO_ELSE	Condition was false → skip to Else branch
JUMP_OUT_WHILE	Loop done or break → exit While
JUMP_BACK_WHILE	Loop body complete → return to While header

PYTHON ROLE — deterministic structure navigation

The Python engine receives the LLM token and computes the exact next line:

- → Bracket matching: maps If/Else/While nesting to line indices.
- → ADVANCE_1 → simply increment line by 1.
- → JUMP_TO_ELSE → find the matching Else/} brace, move ★ there.
- → JUMP_BACK_WHILE → find the While header, move ★ back.
- → Edge cases: ERROR result → stay on same line; break → exit loop.

```
LLM token: JUMP_TO_ELSE
Python:    find_matching_else(line=2, program)
           → Else at line 5 → move ★ to line 5
```

v3 outcome: Control flow is fully separated — the LLM makes a high-level decision, Python handles the structural navigation. No module is overloaded.

Execution Trace · Initialization

Setting up the loop variable before iteration begins

Step 0

NL PROGRAM

```
★ row = 1
While (True) {
    If ($row > $grid_height) {
        break
    }
    Else {
        color_1 = <color of first cell in row $row>
        If ($color_1 != 0) {
            color_2 = <color of last cell in row $row>
            left = '2'
            right = '10'
            While (True) {
                If ($left >= 6) { break }
                Else {
                    grid[$row][$left] = $color_1
                    grid[$row][$right] = $color_2
                    left = $left + 1
                    right = $right - 1
                }
            }
            grid[$row][6] = 5
        }
        Else { }
        row = $row + 1
    }
}
return
```

Interpreter

```
set_local(name='row', value='1')
```

Executor

```
Python path:
  local_vars['row'] = 1
```

PC Updater

ADVANCE_1

LOCAL MEMORY

row = 1

GLOBAL MEMORY

grid (5×11) grid_width=11 grid_height=5

GRID STATE

4										8
6										9

Program begins. Row counter initialized to 1 (1-indexed). Grid unchanged.

Execution Trace · Discovering a Colored Row

LLM inspects the grid to find the color of the first cell in row 2

Step 11

NL PROGRAM

```
row = 1
While (True) {
    If ($row > $grid_height) {
        break
    }
    Else {
★       color_1 = <color of first cell in row $row>
        If ($color_1 != 0) {
            color_2 = <color of last cell in row $row>
            left = '2'
            right = '10'
            While (True) {
                If ($left >= 6) { break }
                Else {
                    grid[$row][$left] = $color_1
                    grid[$row][$right] = $color_2
                    left = $left + 1
                    right = $right - 1
                }
            }
            grid[$row][6] = 5
        }
        Else { }
        row = $row + 1
    }
}
return
```

Interpreter

```
eval_query(dst='color_1',
  query='color of first cell in row $row')
```

Executor

```
LLM path (spatial query):
  grid[1][0] = 4
```

PC Updater

ADVANCE_1

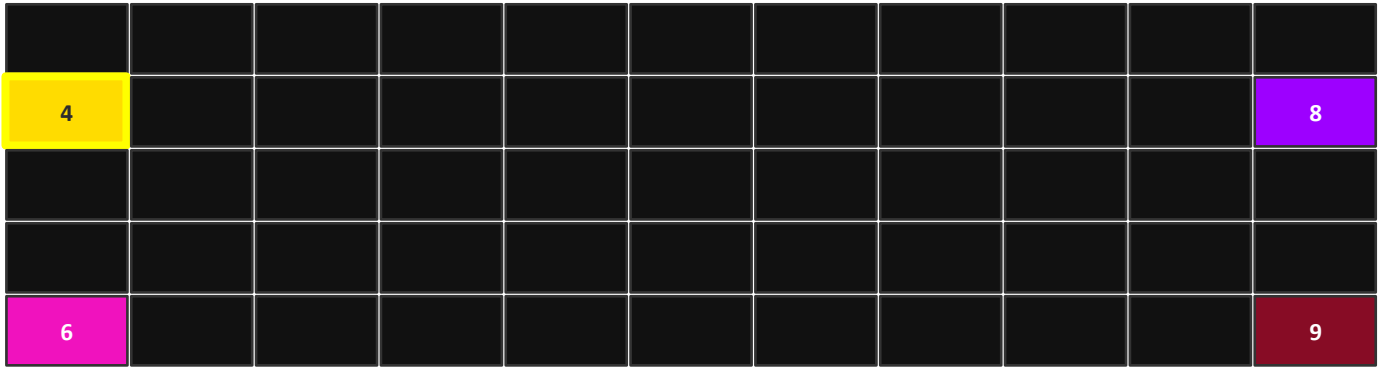
LOCAL MEMORY

```
row = 2
color_1 = → 4 (yellow)
```

GLOBAL MEMORY

grid (5×11) grid_width=11 grid_height=5

GRID STATE



LLM reads grid[1][0] = 4 (yellow). Row 2 is non-black → enters the fill branch.

Execution Trace · First Grid Mutation

Python executor writes color_1 into grid — the core solving step

Step 18

NL PROGRAM

```
row = 1
While (True) {
    If ($row > $grid_height) {
        break
    }
    Else {
        color_1 = <color of first cell in row $row>
        If ($color_1 != 0) {
            color_2 = <color of last cell in row $row>
            left = '2'
            right = '10'
            While (True) {
                If ($left >= 6) { break }
                Else {
★                   grid[$row][$left] = $color_1
                    grid[$row][$right] = $color_2
                    left = $left + 1
                    right = $right - 1
                }
            }
            grid[$row][6] = 5
        }
        Else { }
        row = $row + 1
    }
}
return
```

Interpreter

```
grid_set(x='$row', y='$left',
         rhs='$color_1')
```

Executor

```
Python path (exact):
  grid[1][1] = 4
```

PC Updater

ADVANCE_1

LOCAL MEMORY

```
row = 2
color_1 = 4
color_2 = 8
left = 2
right = 10
```

GLOBAL MEMORY

grid (5×11) grid_width=11 grid_height=5

GRID STATE

4	4									8
6										9

grid_set runs in Python — no LLM needed. Two cells written exactly. Inner loop continues.

Execution Trace · Row Filled + Midpoint Set

Inner loop finished; midpoint cell set to gray (5)

Step 30

NL PROGRAM

```
row = 1
While (True) {
    If ($row > $grid_height) {
        break
    }
    Else {
        color_1 = <color of first cell in row $row>
        If ($color_1 != 0) {
            color_2 = <color of last cell in row $row>
            left = '2'
            right = '10'
            While (True) {
                If ($left >= 6) { break }
                Else {
                    grid[$row][$left] = $color_1
                    grid[$row][$right] = $color_2
                    left = $left + 1
                    right = $right - 1
                }
            }
★           grid[$row][6] = 5
        }
        Else { }
        row = $row + 1
    }
}
return
```

Interpreter

```
grid_set(x='$row', y=6, rhs=5)
```

Executor

```
Python path (exact):
  grid[1][5] = 5  (gray midpoint)
```

PC Updater

ADVANCE_1

LOCAL MEMORY

```
row = 2
color_1 = 4
color_2 = 8
left = 6
right = 5
```

GLOBAL MEMORY

grid (5×11) grid_width=11 grid_height=5

GRID STATE

4	4	4	4	4	5	8	8	8	8	8
6										9

Row 2 fully transformed.

Execution Trace · Execution Complete

All rows processed — return statement reached

Step 122

NL PROGRAM

```
row = 1
While (True) {
    If ($row > $grid_height) {
        break
    }
    Else {
        color_1 = <color of first cell in row $row>
        If ($color_1 != 0) {
            color_2 = <color of last cell in row $row>
            left = '2'
            right = '10'
            While (True) {
                If ($left >= 6) { break }
                Else {
                    grid[$row][$left] = $color_1
                    grid[$row][$right] = $color_2
                    left = $left + 1
                    right = $right - 1
                }
            }
            grid[$row][6] = 5
        }
        Else { }
        row = $row + 1
    }
}
★ return
```

Interpreter

```
return()
```

Executor

```
Terminal action:
  state['terminated'] = True
```

PC Updater

RETURN

LOCAL MEMORY

…
terminated = True

GLOBAL MEMORY

grid (5×11) grid_width=11 grid_height=5

GRID STATE

4	4	4	4	4	5	8	8	8	8	8
6	6	6	6	6	5	9	9	9	9	9

122 steps, exact match with expected output ✓ — both colored rows filled, midpoints placed.

Preliminary Results

TASK EXECUTION SUMMARY

Task ID	Task Type	Steps	LLM Calls	Exact Match
29c11459	Row fill + midpoint	122	300	✓ Yes
2204b7a8	Color mapping	118	295	✓ Yes
5582e5ca	Object transformation	81	207	✓ Yes

KEY OBSERVATIONS

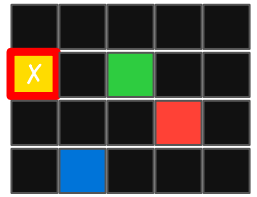
- Structured NL programs execute correctly on all tested tasks with exact-match output.
- Average ~2.5 LLM calls per step.
- Deterministic Python path handles ~70% of actions; LLM path used only for spatial queries.
- Programs are currently hand-authored — automatic generation is future work.

Failure Modes: Where Things Go Wrong

Four categories of LLM errors observed during execution

1 Wrong Reading

LLM misreads a cell's color value from the grid



LLM: 3 (green)

Actual: 4 (yellow)

```
eval_query(dst='color_1',
  query='color of cell [1][0]')
```

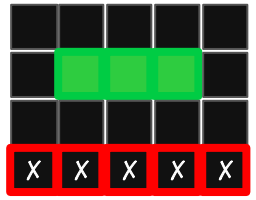
```
LLM returns:  color_1 = 3
Actual grid:  grid[1][0] = 4
```

IMPACT:

Downstream writes use wrong color → row filled green instead of yellow.

2 Wrong Indexing

LLM returns the wrong row/column index for a spatial query



← row 1: correct

← row 3: LLM target

```
eval_query(dst='row_idx',
  query='row having color 3')
```

```
LLM returns:  row_idx = 3
Actual:  color 3 at row 1
```

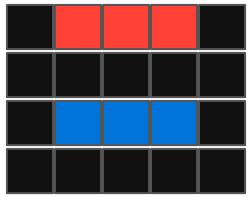
IMPACT:

Operations target wrong row → mutations land in empty/incorrect cells.

3 Wrong Evaluation

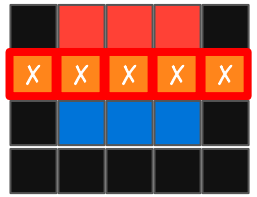
LLM evaluates a boolean condition incorrectly

BEFORE



→

AFTER:




```
eval_condition(
  cond='$color_1 != 0')
```

```
State:  color_1 = 0
LLM says True (should be False)
```

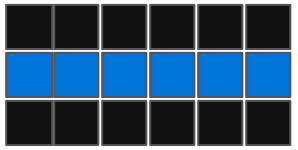
IMPACT:

Enters fill branch for skip-row → writes garbage into an all-black row.

4 PC Updater Error

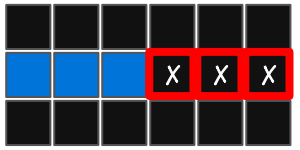
LLM outputs the wrong control-flow token

EXPECTED:



vs

ACTUAL:




```
__cond = False
If ($left >= 6) { break }
```

```
LLM: ADVANCE_1 (→ break)
Should be: JUMP_TO_ELSE
```

IMPACT:

Loop exits early → row only half-filled. Or loop never exits → timeout.

Mitigations: Python path eliminates errors for grid_set, eval_expr, set_local. Remaining errors motivate future work on stronger spatial reasoning.

Future Directions & Conclusion

FUTURE WORK

→ Automatic NL program generation from training pairs (currently hand-authored — the key bottleneck)

→ Scale evaluation to full ARC-AGI-1 (400 eval tasks) and ARC-AGI-2 benchmark

→ Improve executor reliability

KEY TAKEAWAYS

- Structured NL programs bridge the gap between less expressive python code and unverifiable free-form NL.
- Hybrid execution (Python + LLM) achieves exact-match correctness with step-by-step transparency.
- The three-component architecture (Interpreter, Executor, PC Updater) cleanly separates concerns.
- Preliminary results demonstrate feasibility on multiple ARC task types.

"From Prompts to Programs" — making LLM reasoning executable, verifiable, and transparent.

Thank you · Questions?